MilestoneDelivery: Vision-IntegratedRobotics

Godlov Tabow Manfouo

Mert Geceli

Abstract

This report presents the progress of a vision-integrated robotic cell milestone. The system successfully establishes a communication bridge between a Baumer industrial camera and a KUKA KRC4 controller using a custom C++ application and the EthernetKRL protocol. While target identification and coordinate mapping via homography have been implemented, current results indicate that the calibration requires further refinement to achieve industrial-grade precision. Consequently, this delivery focuses on the validation of the communication handshake and the operational logic. Future iterations will focus on optimizing the transformation matrix and fine-tuning the Z-axis operational heights for actual material handling.

1 Introduction

This milestone focuses on the development of a vision-guided system for a KUKA robotic arm. The technical framework is built upon three pillars:

1. Visual Perception: Image acquisition using the NeoAPI SDK and processing via OpenCV to detect specific geometries (black cubes) through binary thresholding.
2. Spatial Transformation: The implementation of homography to translate pixel-based detections into the robot's Cartesian coordinate system, accounting for perspective and scaling offsets.
3. Synchronized Communication: A master-slave TCP/IP architecture using the EthernetKRL (EKI) protocol, allowing for high-speed data exchange and binary packet casting.

2 Vision System Implementation: The C++ Framework

The core intelligence of this robotic cell resides in a custom C++ application developed within the Code::Blocks environment. This framework manages the synchronization between visual perception and robotic motion by handling two distinct network interfaces.

2.1 Network Configuration and IP Addressing

To establish a reliable communication pipeline, the PC is configured with a static IP architecture that segments industrial traffic. The system operates under the following addressing scheme:

- Vision Subnet: The Baumer industrial camera is assigned the IP address 192.168.41.65. The PC connects to this device via a dedicated interface to ensure high-bandwidth image transfer without interference.
- Robot Subnet: The KUKA KRC4 controller acts as a server at IP 192.168.41.64. The C++ application connects to this address via port 54600 using standard TCP/IP sockets to dispatch motion commands.

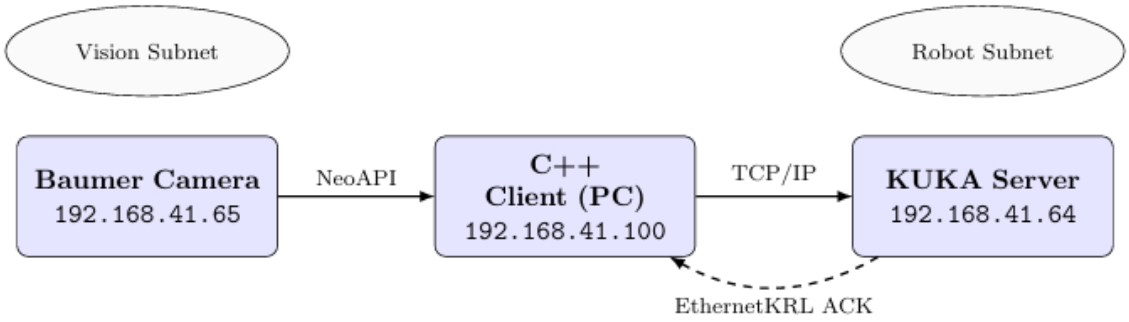


Figure 1: System Communication Topology.

2.2 Camera Interface and Data Acquisition

The system implements the NeoAPI SDK to communicate with the Baumer industrial camera. The following segment illustrates the initialization and frame retrieval process:

```
    //Targeted Camera IP within the Vision Subnet
    Cam camera;
    string camIdentifier = "192.168.41.65";

    try {
        while(!camera.IsConnected()) {
            camera.Connect(NeoString(camIdentifier.c_str()))
        }

        // --- CAMERA CONFIGURATION ---
        camera.StopStreaming();

        // Continuous capture mode
        camera.f().AcquisitionMode.Set(AcquisitionMode::Cont

        // Auto exposure - adapts to lighting
        camera.f().ExposureAuto.Set(ExposureAuto::Continuous

        // Frame rate = 1 FPS (stable, avoids motion blur)
        camera.f().AcquisitionFrameRateEnable.Set(true);
        camera.f().AcquisitionFrameRate.Set(1);

        camera.SetImageBufferCount(1);
        camera.SetImageBufferCycleCount(1);
```

2.3 Image Processing Pipeline and Target Identification

Once the frame is captured, the system identifies the target cube through a subtraction-based segmentation strategy. By calculating the absolute difference between a static reference and the live feed, we isolate new geometry in the workspace. The identification is further refined using binary thresholding and morphological operations to eliminate noise. The logic used to extract the object's centroid and orientation is implemented as follows:

```

    // Threshold converts to binary image
    // OTSU automatically selects optimal threshold
    // 30 base threshold
    // low value - more pixels = white (noise)
    // high value - fewer pixels = white (may miss object)
    // Too much noise: increase value
    // Object disappears: decrease value
    threshold(diff, bin, 30, 255, THRESH_BINARY | THRESH_OTSU);

    // Morphological filtering
    // Removes noise and fills gaps
    // Noise remains: increase kernel
    // Object breaks apart: decrease kernel
    Mat kernel = getStructuringElement(MORPH_RECT, Size(9,9));

    morphologyEx(bin, bin, MORPH_OPEN, kernel);  // remove small noise
    morphologyEx(bin, bin, MORPH_CLOSE, kernel); // close holes

```

```
// --- CONTOUR DETECTION ---
vector<vector<Point>> contours;
findContours(bin, contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);

for (size_t i = 0; i < contours.size(); i++) {

    double area = contourArea(contours[i]);

    // Filter small objects/Ignore small objects
    // Too many small detections: increase threshold
    // Object not detected: decrease threshold
    if (area < 2000) continue;

    // Mask for intensity check
    Mat mask = Mat::zeros(cur.size(), CV_8UC1);
    drawContours(mask, contours, (int)i, Scalar(255), -1);

    // Reject bright objects (cube is dark)
    // Cube not detected: increase threshold
    // Bright noise detected: decrease threshold
    if (mean(cur, mask)[0] > 120) continue;
```

```

    // Shape compactness filter
    // Object rejected: decrease threshold
    // Noise passes: increase threshold
    Rect bbox = boundingRect(contours[i]);
    if ((area / bbox.area()) < 0.5) continue;

    // Aspect ratio filter
    RotatedRect box = minAreaRect(contours[i]);
    float ratio = max(box.size.width, box.size.height) /
                  min(box.size.width, box.size.height);

    // Reject long/thin objects
    // Cube rejected: increase limit
    // Wrong objects pass: decrease limit
    if (ratio > 2.0) continue;


```

2.4 Calibration

To translate image coordinates (x,y) from the Baumer sensor into the robot's Cartesian space (X,Y), the system utilizes a homography. This mathematical model accounts for rotation, translation, and scaling offsets between the camera's perspective and the robot's base frame. The transformation matrix is derived using the findHomography(camPts, robotPts, RANSAC) function. This function calculates a 3 × 3 matrix from three corresponding points identified in both coordinate systems.

```cpp
// --- CALIBRATION (HOMOGRAPHY) ---
// Maps camera pixels to robot coordinates
vector<Point2f> camPts = {
    Point2f(646, 248),
    Point2f(617, 770),
    Point2f(1395, 815),
    Point2f(1441, 286) // 4. Point: on the workspace, the corner without a circle, should be detected correctly
};
// take the left gripper as the center
// in mm
vector<Point2f> robotPts = {
    Point2f(688.00f, 164.12f),
    Point2f(688.00f, 356.29f),
    Point2f(396.70f, 356.29f),
    Point2f(396.70f, 161.66f) // 4. Point: on the workspace, the corner without a circle, should be detected correctly
};

// RANSAC improves robustness against bad calibration points
Mat H = findHomography(camPts, robotPts, RANSAC);
```

2.4.1 Geometric Interpretation

The mapping follows the linear model where pixel coordinates are transformed into robot workspace units (mm). This ensures that any detected centroid is accurately projected before dispatching motion commands to the KUKA server. The relationship is expressed by the following matrix multiplication:

$$\begin{bmatrix} X \\ Y \\ W \end{bmatrix} = H \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

$$X = h_{11}x + h_{12}y + h_{13}$$
$$Y = h_{21}x + h_{22}y + h_{23}$$
$$W = h_{31}x + h_{32}y + h_{33}$$

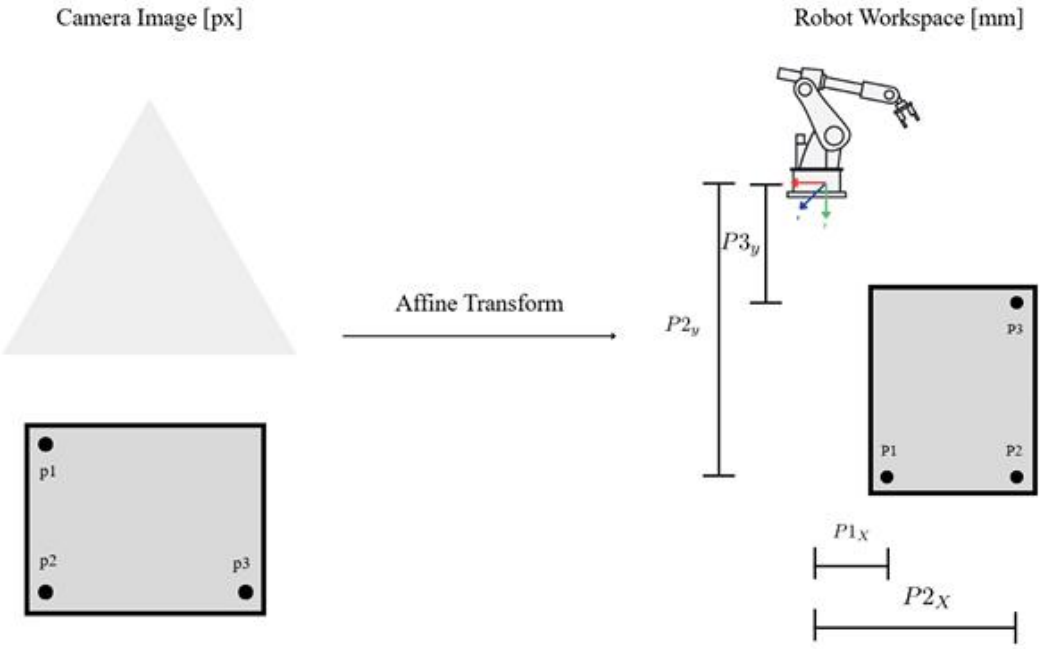
$$X' = \frac{X}{W}, \quad Y' = \frac{Y}{W}$$

Where:

- x,y: pixel coordinates
- X,Y: transformed coordinates
- W: perspective scaling factor
- H: transformation matrix

2.5 TCP/IP Communication and Command Dispatching

The interface with the KUKA controller is established via a TCP socket on port 54600, utilizing the EthernetKRL (EKI) XML configuration. The protocol implements a synchronous master slave architecture where the C++ application acts as the client and the KUKA controller as the server.



2.5.1 Command Classification

The communication protocol distinguishes between two primary data types:

- String-Based Commands: Plain text identifiers used to trigger discrete subroutines within the KRL loop. These include:
  - "home": Moves the robot to the initial XHOME position.
  - "open": Triggers the gripper to release the object.
  - "close": Triggers the gripper to secure the object.
- Binary Packets ("move"): A hybrid 64-byte buffer containing a 4-byte string header followed by 24 bytes of floating-point spatial data.

2.5.2 Data Packet Structure

For motion commands, the system packages coordinates into a raw byte stream. The CAST FROM function on the robot side reconstructs the original E6POS values from this binary sequence.

2.5.3 Operational Pick & Place Sequence

The system executes a deterministic handshake for every action. The C++ client follows this sequence to ensure mechanical safety:

1. Handshake Initial: Send "home" → Wait for ACK.
2. Gripper Prep: Send "open" → Wait for ACK.
3. Approach: Send "move" (Target X,Y, High Z) → Wait for ACK.
4. Pick: Send "move" (Target X,Y, Low Z) → Wait for ACK → Send "close".
5. Retract: Send "move" (Target X,Y, High Z) → Wait for ACK. 5
6. Place: Send "move" (Place X,Y,Z) → Wait for ACK → Send "open".

```cpp
void mandarMovimiento(int sock, float x, float y, float z, float a, float b, float c) {
    char buffer[BUFFER_SIZE];
    memset(buffer, 0, BUFFER_SIZE);

    memcpy(buffer, "move", 4); // header identifier

    int offset = 4;

    // Copy each float into buffer sequentially
    memcpy(buffer + offset, &x, sizeof(float)); offset += sizeof(float);
    memcpy(buffer + offset, &y, sizeof(float)); offset += sizeof(float);
    memcpy(buffer + offset, &z, sizeof(float)); offset += sizeof(float);
    memcpy(buffer + offset, &a, sizeof(float)); offset += sizeof(float);
    memcpy(buffer + offset, &b, sizeof(float)); offset += sizeof(float);
    memcpy(buffer + offset, &c, sizeof(float));

    send(sock, buffer, BUFFER_SIZE, 0);

    // Wait for acknowledgment - ensures safe sequential execution
    memset(buffer, 0, BUFFER_SIZE);
    read(sock, buffer, BUFFER_SIZE);
```

2.5.4 Operational Control Logic and Command Sequence

The execution of a complete Pick & Place cycle is managed by a state machine within the C++ application. The following sequence is executed once a target is successfully identified by the vision system:

1. Synchronization: The system initializes communication and forces a movement to the XHOME position to establish a known safe starting point.
2. Preparation: The "open" command is sent to actuate the gripper's release state.
3. Approach: A binary "move" packet is dispatched with the target's X,Y coordinates but with a high Z value (Z APPROACH PICK) to align the gripper safely above the object.

4. Acquisition: The robot descends to Z PICK, the "close" command is sent to secure the cube, and the robot retracts to the approach height.

5. Delivery: The robot moves toward the predefined PLACE coordinates using a secondary approach height to ensure a smooth transition.

6. Release: Upon reaching the destination, the "open" command is triggered, followed by a final return to "home".

This procedural approach ensures that the robot never receives a new coordinate until the previous motion has been fully completed and acknowledged by the KRC4 controller.

3 Robot-Side Configuration and Logic

The integration between the vision system and the physical execution is managed by the KRC4 controller. The robot operates as a server, remaining in a passive state until the C++ client initi ates a connection. This architecture requires a communication interface (XML) and a processing script (KRL).

3.1 EthernetKRL Interface Configuration (XML)

The communication channel is defined by an XML schema that maps network data into system f lags and memory buffers. This ensures that the robot can interpret the 64-byte packets sent from the PC. Storage Path: C:\KRC\Roboter\Config\User\Common\EthernetKRL\GripperControl.xml

```
<INTERNAL>
   <IP>192.168.41.64</IP>
   <PORT>54600</PORT>
   <ALIVE Set_Flag="1"/> </INTERNAL>
<RECEIVE>
   <RAW>
      <ELEMENT Tag="Command" Type="BYTE" Set_Flag="2" Size="64"/>
   </RAW>
</RECEIVE>
```

• Flag 1 (ALIVE): Automatically managed by the EthernetKRL (EKI) driver. It remains TRUE as long as the TCP/IP socket with the PC is open.

• Flag 2 (RECEIVE): A critical synchronization trigger. It is activated only when the 64 byte buffer is completely filled, preventing the robot from processing partial or corrupted data packets.

3.2 KRLCommandParsingandMotionControl

The KUKA Robot Language (KRL) script manages the operational loop. Its primary function is to ”parse” the incoming byte array, identifying whether the instruction is a discrete string command or a binary motion packet. StoragePath: R1\Program\movehome.src

```
// --- Synchronization and Wait ---
WAIT FOR $FLAG[1] // Ensure PC client is connected

LOOP
   $FLAG[2] = FALSE // Reset receive flag
   WAIT FOR $FLAG[2] // Wait for new data from PC

   // Retrieve the byte stream into the "Bytes[]" array
   RET=EKI_GetString("GripperControl","Command",Bytes[])

   // 1. String Comparison for Gripper Control
   IF func_String(Bytes[], "open") == TRUE THEN
      GRPg_SetStateAndCheck(1, 1, 0.5, 1) // Actuate Open
   ENDIF

   // 2. Binary Extraction for Cartesian Motion
   IF func_String(Bytes[], "move") == TRUE THEN
      offsetVar = 4 // Skip "move" header (4 bytes)
      // Reconstruct floats from raw bytes into targetPos
      CAST_FROM(Bytes[], offsetVar, targetPos.X, targetPos.Y, targetPos.
         Z, targetPos.A, targetPos.B, targetPos.C)

      PTP targetPos // Execute point-to-point movement
   ENDIF

   // 3. Handshake Acknowledgment (ACK)
   RET = EKI_Send("GripperControl", "Command") // Notify PC of
      completion
ENDLOOP
```

3.2.1 DataReconstruction via CAST_FROM

The C++ system transmits coordinates as raw 32-bitfloating-pointvalues. The CAST_FROM instruction is vital as it interprets these binary sequences, starting after the 4-byte string header (”move”), and reconstructs the X,Y,Z,A,B,C components of the E6POS structure. This method ensures high-precision positioning without the overhead of string-to-float parsing on the robot side.

3.2.2 The Synchronous Handshake

The use of EKI_Send at the end of each loop iteration is fundamental to the system’s stability. By sending back the buffer, the robot confirms the successful completion of the current task. This signal is caught by the read() function in the C++client, effectively

closing the control loop and preventing the PC from sending subsequent commands while the robot is still in motion.

4 Future Improvements and Recommendations

As this report represents a development milestone, several areas have been identified for optimization to transition from a simulated pick-and-place sequence to a fully operational industrial cycle.

4.1 Enhanced Visual Feedback and User Interface (UI)

Currently, the system provides data primarily through the console. To improve operational safety and monitoring, the following features could be implemented:

- Target Visualization: Integration of OpenCV drawing functions (cv::rectangle and cv::putText) to highlight the detected cube and display its calculated coordinates on the live feed before dispatching them to the robot.
- Graphical User Interface (GUI): Development of a dedicated interface using libraries such as Qt or OpenCV HighGUI to allow the operator to confirm or reject a detection before the robot initiates movement.

4.2 Calibration Refinement and Precision Tuning

The current homography serves as a functional baseline but requires optimization to eliminate current positioning errors.

- Iterative Calibration: Implementation of a multi-point calibration routine (more than 4 points) to calculate a more robust transformation matrix and minimize the Mean Squared Error (MSE).
- Parallax Correction: Accounting for the camera's mounting angle to reduce perspective distortion that affects coordinate accuracy at the edges of the workspace.

4.3 Dynamic Z-Axis and Physical Acquisition

The system must transition from the current "simulated pick" to physical material handling.

- Physical Height Validation: Fine-tuning the Z PICK constant to match the physical height of the cube, allowing the gripper to descend to the exact contact point without a collision

- Error Handling: Implementation of timeout routines in the C++ client to handle cases where the robot fails to send an ACK, preventing the application from hanging indefinitely.

4.4 Automated Reference Acquisition

Currently, the system depends on a pre-loaded static reference image. A future improvement includes a ”Set Reference” button in the UI that allows the system to capture a new background image dynamically, adapting to changes in ambient lighting or workspace configuration